

MI - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continuous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste0(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 4369 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 4329 values"
## [1] "40 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	549	33	127	NA	NA	709
2	62	NA	435	96	110	703
3	310	NA	168	209	15	702
4	301	NA	239	65	38	643
5	229	NA	213	72	38	552
6	209	1	135	72	22	439
7	177	NA	105	26	17	325
8	90	NA	53	25	4	172
9	75	NA	2	NA	7	84

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7743300	0.0465444	0.1791255	NA	NA	709
2	0.0881935	NA	0.6187767	0.1365576	0.1564723	703
3	0.4415954	NA	0.2393162	0.2977208	0.0213675	702
4	0.4681182	NA	0.3716952	0.1010886	0.0590980	643
5	0.4148551	NA	0.3858696	0.1304348	0.0688406	552
6	0.4760820	0.0022779	0.3075171	0.1640091	0.0501139	439
7	0.5446154	NA	0.3230769	0.0800000	0.0523077	325
8	0.5232558	NA	0.3081395	0.1453488	0.0232558	172
9	0.8928571	NA	0.0238095	NA	0.0833333	84

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position
```

```
syll_comp_plot <- ggplot(syll_comp_plot_data,
  aes(x=pos,y=percent,group=syll_component,
    color=syll_component,
    linetype = syll_component,
    shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=s
```

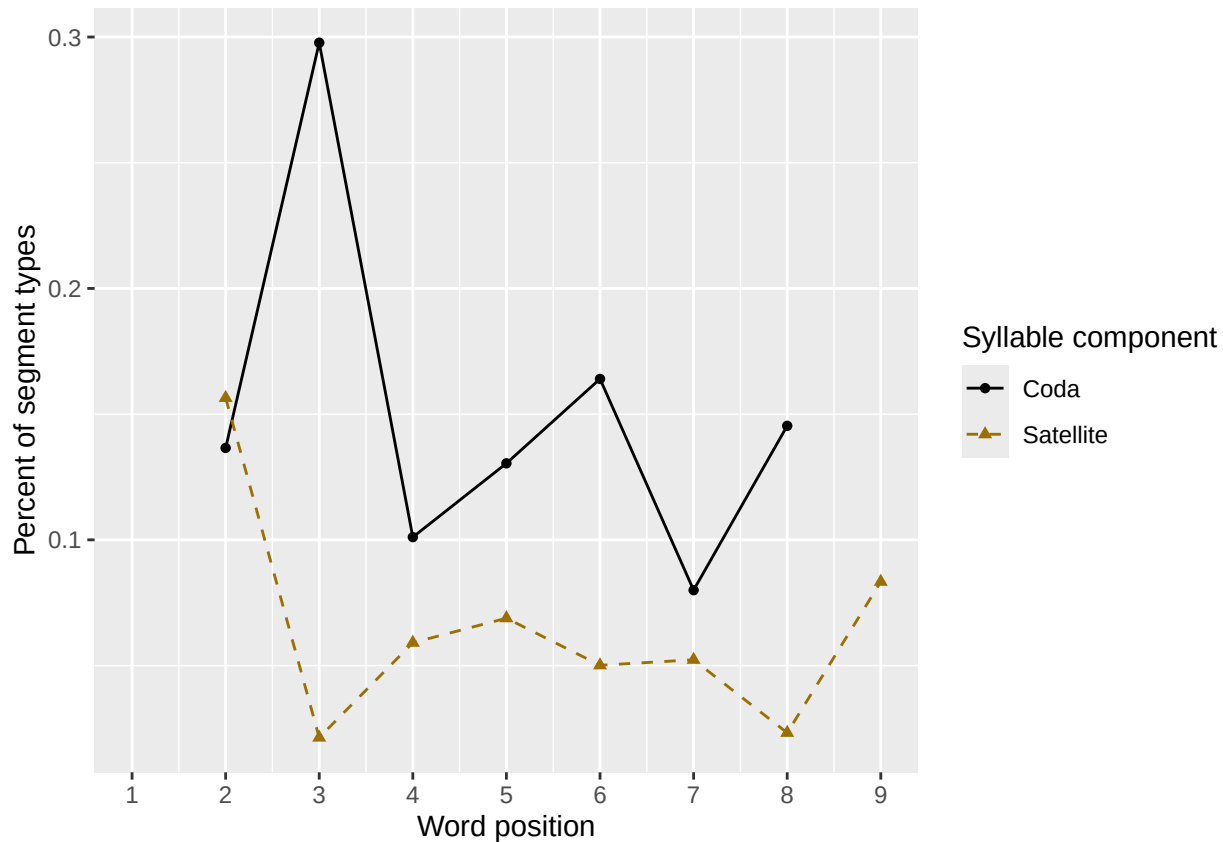
```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
syll_comp_plot
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```



```
# len/pos table
```

```
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
```

```
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
```

```
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.947 0.982 0.930 NA      NA      NA      NA      NA      NA
## 2      5 0.905 0.937 0.926 0.884 NA      NA      NA      NA      NA
## 3      6 0.915 0.889 0.897 0.957 0.855 NA      NA      NA      NA
## 4      7 0.858 0.965 0.911 0.856 0.973 0.912 NA      NA      NA
## 5      8 0.842 0.92  0.933 0.919 0.919 0.940 0.914 NA      NA
## 6      9 0.878 0.920 0.875 0.933 0.889 0.889 0.933 0.9  NA
## 7     10 0.882 0.904 0.831 0.878 0.964 0.882 0.881 0.976 0.881

```

```

# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
"_N_by_len_position.csv"))
pos_len_N_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4    57    57    57    NA    NA    NA    NA    NA    NA
## 2      5    95    95    95    95    NA    NA    NA    NA    NA
## 3      6   117   117   117   117   117    NA    NA    NA    NA
## 4      7   113   113   112   111   112   113    NA    NA    NA
## 5      8   152   150   150   148   149   151   151    NA    NA
## 6      9    90    88    88    90    90    90    90    90    NA
## 7     10    85    83    83    82    84    85    84    82    84

```

```

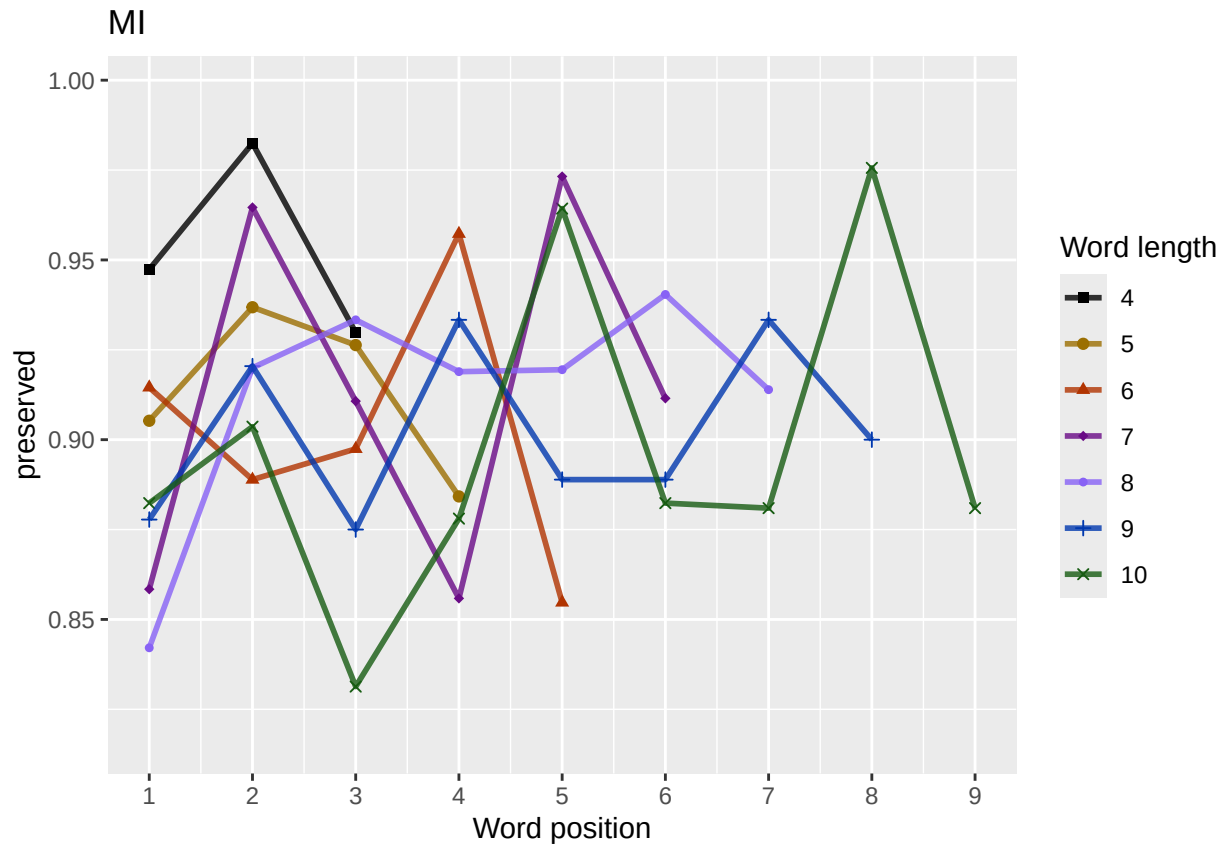
obs_linetypes <- c("solid", "solid", "solid", "solid",
"solid", "solid", "solid", "solid", "solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
pos_len_summary$pos <- factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
paste0(CurPat),
NULL,
c(min_preserved, max_preserved),
palette_values,
shape_values,
obs_linetypes,
pred_linetypes)

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
                      "preserved ~ stimlen",
                      "preserved ~ pos",
                      "preserved ~ stimlen + pos",
                      "preserved ~ stimlen*pos",
                      "preserved ~ I(pos^2)+pos",
                      "preserved ~ stimlen + I(pos^2) + pos",
                      "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## *****
```

```
## model index: 4
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```

##
## Coefficients:
## (Intercept)      stimlen      pos
##      2.68956      -0.07664      0.05245
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4326 Residual
## Null Deviance:      2649
## Residual Deviance: 2642  AIC: 2648
## log likelihood:  -1321.097
## Nagelkerke R2:  0.003344782
## % pres/err predicted correctly:  -717.3995
## % of predictable range [ (model-null)/(1-null) ]:  0.001600198
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos  stimlen:pos
##      3.05742      -0.12110      -0.07062      0.01445
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4325 Residual
## Null Deviance:      2649
## Residual Deviance: 2642  AIC: 2650
## log likelihood:  -1320.751
## Nagelkerke R2:  0.003693774
## % pres/err predicted correctly:  -717.299
## % of predictable range [ (model-null)/(1-null) ]:  0.001739928
## *****
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos
##      2.540576      -0.072151      -0.008941      0.127506
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4325 Residual
## Null Deviance:      2649
## Residual Deviance: 2642  AIC: 2650
## log likelihood:  -1320.77
## Nagelkerke R2:  0.003674285
## % pres/err predicted correctly:  -717.2713
## % of predictable range [ (model-null)/(1-null) ]:  0.001778447
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen

```

```

##      2.69399      -0.05155
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4327 Residual
## Null Deviance:      2649
## Residual Deviance: 2646  AIC: 2650
## log likelihood:  -1323.067
## Nagelkerke R2:  0.00135807
## % pres/err predicted correctly:  -718.1114
## % of predictable range [ (model-null)/(1-null) ]:  0.0006108147
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.296
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4328 Residual
## Null Deviance:      2649
## Residual Deviance: 2649  AIC: 2651
## log likelihood:  -1324.413
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -718.5509
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      3.169e+00      -1.669e-01      -2.234e-02      -4.745e-02      7.115e-05      3.3
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4323 Residual
## Null Deviance:      2649
## Residual Deviance: 2639  AIC: 2651
## log likelihood:  -1319.496
## Nagelkerke R2:  0.004958204
## % pres/err predicted correctly:  -716.8574
## % of predictable range [ (model-null)/(1-null) ]:  0.002353551
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      2.18232      0.03003
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4327 Residual

```



```
## Null Deviance:      2649
## Residual Deviance: 2647 AIC: 2651
## log likelihood: -1323.681
## Nagelkerke R2: 0.000738674
## % pres/err predicted correctly: -718.3029
## % of predictable range [ (model-null)/(1-null) ]: 0.0003446514
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.01182      -0.01282      0.13940
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4326 Residual
## Null Deviance:      2649
## Residual Deviance: 2646 AIC: 2652
## log likelihood: -1322.992
## Nagelkerke R2: 0.001433848
## % pres/err predicted correctly: -718.0566
## % of predictable range [ (model-null)/(1-null) ]: 0.0006869882
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen + pos	2648.194	0.000000	0.000000	0.000000	0.003489	0.000334	28689564	-	0.0524531	NA	NA
preserved ~ stimlen * pos	2649.502	1.307440	0.152010	0.041585	0.700036	938057418	-	-	0.0144513	NA	NA
preserved ~ stimlen + I(pos^2)	2649.540	0.346120	0.510140	0.091555	0.397003	6723540576	-	0.1275059	NA	-	NA
+ pos	2650.135	1.940410	0.379000	0.041155	0.590013	521693990	-	NA	NA	NA	NA
preserved ~ stimlen	2650.822	2.631928	0.268215	0.081770	0.100000	0.00295744	NA	NA	NA	NA	NA

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	2650.992	2.797320	20.2469276	0.752865	0.0049582	168560	-	-	0.0330331	-	7.12e-05
							0.1669018	0.0474492		0.0223361	
preserved ~ pos	2651.362	1.68182	2.205134	0.1062544	0.0000738	7182324	NA	0.0300267	NA	NA	NA
preserved ~ I(pos^2) + pos	2651.984	1.790183	3.1503046	0.458268	0.0014328	111818	NA	0.1393994	NA	-	NA
										0.0128241	

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen + pos"
```

```
print(BestLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      2.68956      -0.07664       0.05245
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4326 Residual
## Null Deviance:      2649
## Residual Deviance: 2642 AIC: 2648
```

```
PosDat$LPFitted<-fitted(BestLPModel)
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.919 0.923 0.927 NA    NA    NA    NA    NA    NA
## 2      5 0.914 0.918 0.922 0.925 NA    NA    NA    NA    NA
## 3      6 0.907 0.912 0.916 0.920 0.924 NA    NA    NA    NA
## 4      7 0.901 0.905 0.910 0.914 0.918 0.922 NA    NA    NA
## 5      8 0.894 0.899 0.903 0.908 0.912 0.916 0.920 NA    NA
## 6      9 0.886 0.891 0.896 0.901 0.906 0.910 0.914 0.918 NA
## 7     10 0.878 0.884 0.889 0.894 0.899 0.904 0.908 0.912 0.916
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
```

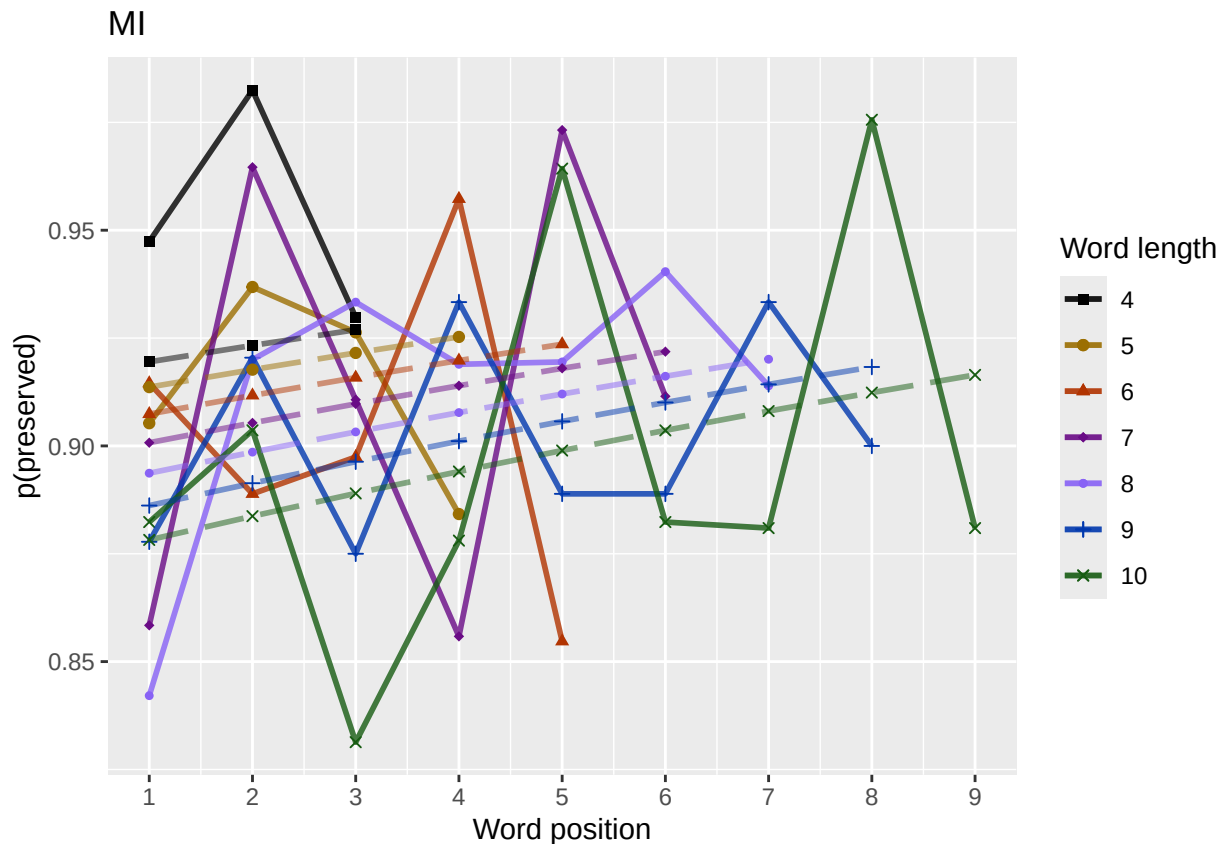
```
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=
```

```
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                                    paste0(PosDat$patient[1]),
                                                    "LPFitted",
                                                    NULL,
                                                    palette_values,
                                                    shape_values,
                                                    obs_linetypes,
                                                    pred_linetypes = c("longdash")
                                                    )

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=
fitted_len_pos_plot
```



length and position without fragments to see if this changes position² influence

```
# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models
```

```
# number responses
resp_num<-0
prev_pos<-9999 # big number to initialize (so first position is smaller)
```

```

resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1       4   709

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 4 / 709 = 0.56 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##    2.67236    -0.07833     0.06555
##
## Degrees of Freedom: 4321 Total (i.e. Null); 4319 Residual
## Null Deviance: 2615
## Residual Deviance: 2607 AIC: 2613
## log likelihood: -1303.482
## Nagelkerke R2: 0.004208669
## % pres/err predicted correctly: -705.5545
## % of predictable range [ (model-null)/(1-null) ]: 0.002008363
## *****
## model index: 5

```

```

##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##      3.02755      -0.12129      -0.05450       0.01411
##
## Degrees of Freedom: 4321 Total (i.e. Null);  4318 Residual
## Null Deviance:      2615
## Residual Deviance: 2606  AIC: 2614
## log likelihood:  -1303.16
## Nagelkerke R2:  0.004535987
## % pres/err predicted correctly:  -705.4629
## % of predictable range [ (model-null)/(1-null) ]:  0.002137832
## *****
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##      2.570112     -0.075284     -0.006248      0.117629
##
## Degrees of Freedom: 4321 Total (i.e. Null);  4318 Residual
## Null Deviance:      2615
## Residual Deviance: 2607  AIC: 2615
## log likelihood:  -1303.329
## Nagelkerke R2:  0.004364595
## % pres/err predicted correctly:  -705.4889
## % of predictable range [ (model-null)/(1-null) ]:  0.002101105
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.15290      0.04286
##
## Degrees of Freedom: 4321 Total (i.e. Null);  4320 Residual
## Null Deviance:      2615
## Residual Deviance: 2612  AIC: 2616
## log likelihood:  -1306.162
## Nagelkerke R2:  0.001480422
## % pres/err predicted correctly:  -706.4952
## % of predictable range [ (model-null)/(1-null) ]:  0.0006796963
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)

```

```

##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      3.254607      -0.173510      -0.002813      -0.140429      -0.001610      0.001610
##
## Degrees of Freedom: 4321 Total (i.e. Null);  4316 Residual
## Null Deviance:      2615
## Residual Deviance: 2605  AIC: 2617
## log likelihood:  -1302.372
## Nagelkerke R2:  0.005337291
## % pres/err predicted correctly:  -705.183
## % of predictable range [ (model-null)/(1-null) ]:  0.00253314
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen
##      2.67977      -0.04744
##
## Degrees of Freedom: 4321 Total (i.e. Null);  4320 Residual
## Null Deviance:      2615
## Residual Deviance: 2613  AIC: 2617
## log likelihood:  -1306.491
## Nagelkerke R2:  0.001145506
## % pres/err predicted correctly:  -706.6131
## % of predictable range [ (model-null)/(1-null) ]:  0.0005130962
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)
##      2.314
##
## Degrees of Freedom: 4321 Total (i.e. Null);  4321 Residual
## Null Deviance:      2615
## Residual Deviance: 2615  AIC: 2617
## log likelihood:  -1307.615
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -706.9764
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      I(pos^2)      pos

```

```
##      2.01781      -0.01033      0.13025
##
## Degrees of Freedom: 4321 Total (i.e. Null);  4319 Residual
## Null Deviance:      2615
## Residual Deviance: 2611  AIC: 2617
## log likelihood:  -1305.733
## Nagelkerke R2:  0.001916904
## % pres/err predicted correctly:  -706.3375
## % of predictable range [ (model-null)/(1-null) ]:  0.0009024724
## *****
```

```
NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                                AIC=NoFrag_LPRes$AIC,
                                row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~	2612.964	0.000000	0.000000	0.000000	0.379995	0.700420	27672363	-	0.0655509	NA	NA
stimlen + pos							0.0783324				
preserved ~	2614.320	0.356493	0.507500	0.0019285	0.0100453	0.60027546	-	-	0.0141125	NA	NA
stimlen * pos							0.1212932	0.0545006			
preserved ~	2614.657	0.693462	0.428810	0.0162947	0.0004362	0.6570113	-	0.1176287	NA	-	NA
stimlen + I(pos^2)							0.0752842			0.0062482	
+ pos											
preserved ~ pos	2616.324	0.360000	0.186378	0.0070820	0.0001480	0.4152902	NA	0.0428630	NA	NA	NA
preserved ~	2616.743	0.780728	0.151016	0.0057385	0.0005333	0.3254607	-	-	0.0406524	-	-
stimlen * (I(pos^2)							0.1735097	0.1404288		0.0028127	0.0016103
+ pos)											
preserved ~	2616.984	0.017532	0.134150	0.0105097	0.0001142	0.5679772	-	NA	NA	NA	NA
stimlen							0.0474361				
preserved ~ 1	2617.220	0.265700	0.118498	0.0045028	0.0000000	0.0313578	NA	NA	NA	NA	NA
preserved ~	2617.467	0.502924	0.105245	0.0039999	0.0001910	0.0917814	NA	0.1302519	NA	-	NA
I(pos^2) + pos										0.0103261	

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
```

```
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.919 0.923 0.928 NA      NA      NA      NA      NA      NA
## 2      5 0.913 0.918 0.923 0.927 NA      NA      NA      NA      NA
## 3      6 0.906 0.912 0.917 0.922 0.926 NA      NA      NA      NA
## 4      7 0.899 0.905 0.911 0.916 0.921 0.925 NA      NA      NA
## 5      8 0.892 0.898 0.904 0.910 0.915 0.920 0.924 NA      NA
## 6      9 0.884 0.891 0.897 0.903 0.908 0.914 0.919 0.924 NA
## 7     10 0.876 0.883 0.890 0.896 0.902 0.907 0.913 0.918 0.923
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
```

```
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
```

```
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted
```

```
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas
```

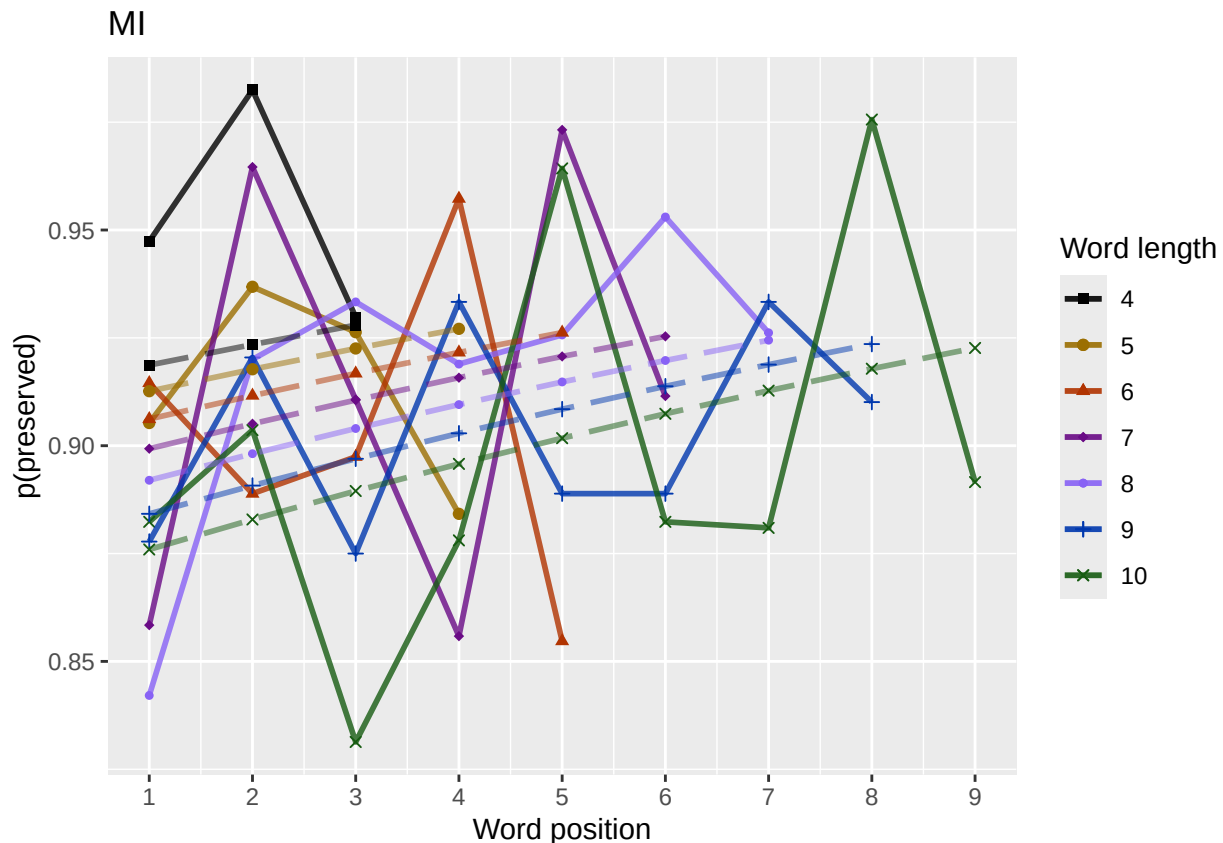
```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.82 - 1.00"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] -0.006250202
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] 0.004306082
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
    2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "No U-shape in this participant"
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentage_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentage_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "no U-shape in this participant"

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small sample sizes)
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##

```

```

## Coefficients:
## (Intercept)          pos      log_freq
##      2.16048      0.03986      0.10731
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4326 Residual
## Null Deviance:      2649
## Residual Deviance: 2633  AIC: 2639
## log likelihood:  -1316.304
## Nagelkerke R2:  0.008170004
## % pres/err predicted correctly:  -715.7991
## % of predictable range [ (model-null)/(1-null) ]:  0.0038244
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos      log_freq
##      2.47103      -0.04643      0.05221      0.09737
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4325 Residual
## Null Deviance:      2649
## Residual Deviance: 2631  AIC: 2639
## log likelihood:  -1315.43
## Nagelkerke R2:  0.009049625
## % pres/err predicted correctly:  -715.4944
## % of predictable range [ (model-null)/(1-null) ]:  0.004247779
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq  pos:log_freq
##      2.17527      0.03481      0.14934      -0.01144
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4325 Residual
## Null Deviance:      2649
## Residual Deviance: 2632  AIC: 2640
## log likelihood:  -1315.934
## Nagelkerke R2:  0.008542812
## % pres/err predicted correctly:  -715.6284
## % of predictable range [ (model-null)/(1-null) ]:  0.004061547
## *****
## model index:  13
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos      log_freq
##      2.314320     -0.041651     -0.009384      0.130933      0.097648

```

```

##
## Degrees of Freedom: 4328 Total (i.e. Null); 4324 Residual
## Null Deviance: 2649
## Residual Deviance: 2630 AIC: 2640
## log likelihood: -1315.072
## Nagelkerke R2: 0.009409205
## % pres/err predicted correctly: -715.3382
## % of predictable range [ (model-null)/(1-null) ]: 0.004464952
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq
## 2.45943 -0.04631 0.19862 0.05228 -0.01297
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4324 Residual
## Null Deviance: 2649
## Residual Deviance: 2630 AIC: 2640
## log likelihood: -1315.138
## Nagelkerke R2: 0.009342814
## % pres/err predicted correctly: -715.4157
## % of predictable range [ (model-null)/(1-null) ]: 0.004357271
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq pos:log_freq
## 2.466523 -0.043917 0.047316 0.133836 -0.009777
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4324 Residual
## Null Deviance: 2649
## Residual Deviance: 2630 AIC: 2640
## log likelihood: -1315.159
## Nagelkerke R2: 0.009321661
## % pres/err predicted correctly: -715.3678
## % of predictable range [ (model-null)/(1-null) ]: 0.004423766
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq
## 2.47520 -0.02142 0.09755
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4326 Residual
## Null Deviance: 2649

```

```

## Residual Deviance: 2635 AIC: 2641
## log likelihood: -1317.376
## Nagelkerke R2: 0.007092431
## % pres/err predicted correctly: -716.2279
## % of predictable range [ (model-null)/(1-null) ]: 0.003228436
## *****
## model index: 11
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq I(pos^2) pos stimlen:log
## 2.29599 -0.04131 0.20327 -0.00975 0.13405 -0.
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4323 Residual
## Null Deviance: 2649
## Residual Deviance: 2630 AIC: 2642
## log likelihood: -1314.753
## Nagelkerke R2: 0.009729817
## % pres/err predicted correctly: -715.2471
## % of predictable range [ (model-null)/(1-null) ]: 0.004591571
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq log_fr
## 2.459416 -0.044621 0.200155 0.048826 -0.009782 -0.
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4323 Residual
## Null Deviance: 2649
## Residual Deviance: 2630 AIC: 2642
## log likelihood: -1315.014
## Nagelkerke R2: 0.009467502
## % pres/err predicted correctly: -715.3436
## % of predictable range [ (model-null)/(1-null) ]: 0.004457433
## *****
## model index: 9
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos log_freq I(pos^2):log_freq
## 2.032157 -0.010632 0.126268 0.206923 0.004172
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4323 Residual
## Null Deviance: 2649
## Residual Deviance: 2630 AIC: 2642
## log likelihood: -1315.042
## Nagelkerke R2: 0.009439145

```

```

## % pres/err predicted correctly: -715.2541
## % of predictable range [ (model-null)/(1-null) ]: 0.004581851
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq stimlen:log_freq
## 2.46374 -0.02127 0.19787 -0.01285
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4325 Residual
## Null Deviance: 2649
## Residual Deviance: 2634 AIC: 2642
## log likelihood: -1317.089
## Nagelkerke R2: 0.007380626
## % pres/err predicted correctly: -716.1507
## % of predictable range [ (model-null)/(1-null) ]: 0.003335718
## *****
## model index: 12
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq I(pos
## 2.320466 -0.039163 -0.008425 0.118852 0.195454
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4322 Residual
## Null Deviance: 2649
## Residual Deviance: 2629 AIC: 2643
## log likelihood: -1314.447
## Nagelkerke R2: 0.0100376
## % pres/err predicted correctly: -715.0594
## % of predictable range [ (model-null)/(1-null) ]: 0.004852333
## *****
## model index: 10
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq I(pos^2) pos stim
## 2.315630 -0.040156 0.288658 -0.008136 0.118494
## log_freq:pos
## -0.050578
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4321 Residual
## Null Deviance: 2649
## Residual Deviance: 2628 AIC: 2644
## log likelihood: -1314.22
## Nagelkerke R2: 0.01026547
## % pres/err predicted correctly: -715.0112

```



```

## % of predictable range [ (model-null)/(1-null) ]: 0.004919304
## *****
## model index: 17
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##    2.68956    -0.07664     0.05245
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4326 Residual
## Null Deviance:      2649
## Residual Deviance: 2642 AIC: 2648
## log likelihood: -1321.097
## Nagelkerke R2: 0.003344782
## % pres/err predicted correctly: -717.3995
## % of predictable range [ (model-null)/(1-null) ]: 0.001600198
## *****
## model index: 18
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos stimlen:pos
##    3.05742    -0.12110    -0.07062     0.01445
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4325 Residual
## Null Deviance:      2649
## Residual Deviance: 2642 AIC: 2650
## log likelihood: -1320.751
## Nagelkerke R2: 0.003693774
## % pres/err predicted correctly: -717.299
## % of predictable range [ (model-null)/(1-null) ]: 0.001739928
## *****
## model index: 20
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##    2.540576    -0.072151    -0.008941     0.127506
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4325 Residual
## Null Deviance:      2649
## Residual Deviance: 2642 AIC: 2650
## log likelihood: -1320.77
## Nagelkerke R2: 0.003674285
## % pres/err predicted correctly: -717.2713
## % of predictable range [ (model-null)/(1-null) ]: 0.001778447
## *****
## model index: 15

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      2.69399      -0.05155
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4327 Residual
## Null Deviance:      2649
## Residual Deviance: 2646 AIC: 2650
## log likelihood: -1323.067
## Nagelkerke R2:  0.00135807
## % pres/err predicted correctly: -718.1114
## % of predictable range [ (model-null)/(1-null) ]:  0.0006108147
## *****
## model index:  14
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.296
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4328 Residual
## Null Deviance:      2649
## Residual Deviance: 2649 AIC: 2651
## log likelihood: -1324.413
## Nagelkerke R2:  0
## % pres/err predicted correctly: -718.5509
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  21
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      3.169e+00      -1.669e-01      -2.234e-02      -4.745e-02      7.115e-05      3.3
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4323 Residual
## Null Deviance:      2649
## Residual Deviance: 2639 AIC: 2651
## log likelihood: -1319.496
## Nagelkerke R2:  0.004958204
## % pres/err predicted correctly: -716.8574
## % of predictable range [ (model-null)/(1-null) ]:  0.002353551
## *****
## model index:  16
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)

```

```
##
## Coefficients:
## (Intercept)          pos
##      2.18232      0.03003
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4327 Residual
## Null Deviance:      2649
## Residual Deviance: 2647  AIC: 2651
## log likelihood:  -1323.681
## Nagelkerke R2:  0.000738674
## % pres/err predicted correctly:  -718.3029
## % of predictable range [ (model-null)/(1-null) ]:  0.0003446514
## *****
## model index:  19
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.01182      -0.01282      0.13940
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4326 Residual
## Null Deviance:      2649
## Residual Deviance: 2646  AIC: 2652
## log likelihood:  -1322.992
## Nagelkerke R2:  0.001433848
## % pres/err predicted correctly:  -718.0566
## % of predictable range [ (model-null)/(1-null) ]:  0.0006869882
## *****
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                           AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                       by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                              RunLabel,"_",CurTask,
                              "_freq_len_pos_model_summary.csv"),row.names = FALSE)

kable(FLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	log_freq	log_pos	log_freq	I(pos^2)	log_freq	I(pos^2)	log_freq	I(pos^2)	log_freq	I(pos^2)
preserved ~	2638.6000	0.0000	1.0000	0.9839	0.1700	NA	0.1073	NA	0.0398	NA	NA	NA	NA	NA	NA	NA
pos +																
log_freq																

Model	AIC Delta	AIC	AICw	NagR ²	Intercept	log_stimlen	log_freq	log_pos	log_freq(I(pos^2))	pos^2	log_freq(I(pos^2))	len:I(pos^2)
preserved ~ stimlen + pos + log_freq	2638.825	0.00588	2640.004	0.892	0.1027	0.0973738	0.0522133	NA	NA	NA	NA	NA
					0.0464340							
preserved ~ pos * log_freq	2639.867	0.00533	2640.012	0.738	0.1493142	0.0348126	0.0114417	NA	NA	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos + log_freq	2640.154	0.01464	2640.092	0.901	0.1320	0.0970116	0.1309330	NA	-	NA	NA	NA
					0.0416505				0.0093836			
preserved ~ stimlen * log_freq + pos	2640.276	0.01403	2640.086	0.903	0.1435	0.1986203	0.0522811	NA	NA	NA	NA	NA
					0.0463120	0.0129733						
preserved ~ stimlen + pos * log_freq	2640.378	0.00485	2640.102	0.225	0.1338338	0.0473159	0.0097775	NA	NA	NA	NA	NA
					0.0439172							
preserved ~ stimlen + log_freq	2640.731	0.02606	2640.067	0.729	0.15201	0.0975113	NA	NA	NA	NA	NA	NA
					0.0214249							
preserved ~ stimlen * log_freq + I(pos^2) + pos	2641.289	0.00834	2640.066	0.729	0.198988	0.2032663	0.1340110	NA	-	NA	NA	NA
					0.0413071	0.0135320			0.0097502			
preserved ~ stimlen * log_freq + pos *	2642.328	0.01891	2640.065	0.901	0.1416	0.2001551	0.0488219	-	NA	NA	NA	NA
					0.0446207	0.0097822		0.0070907				
preserved ~ (I(pos^2) + pos) * log_freq	2642.384	0.01750	2640.089	0.392	0.151	0.2069219	0.1262678	NA	-	0.0041715	NA	NA
									0.0479687	0.0106316		
preserved ~ stimlen * log_freq	2642.378	0.00416	2640.032	0.731	0.173737	0.1978728	NA	NA	NA	NA	NA	NA
					0.0212742	0.0128546						
preserved ~ stimlen + (I(pos^2) + pos) * log_freq	2642.423	0.01461	2640.092	0.911	0.1320	0.1954518	0.1188520	NA	-	0.0043832	NA	NA
					0.0391631		0.0481153		0.0084253			
preserved ~ stimlen * log_freq + (I(pos^2) + pos) * log_freq	2644.543	0.00735	2640.085	0.749	0.15630	0.2886583	0.1184017	-	-	NA	0.0050731	NA
					0.0401559	0.0124143		0.0503700	0.01364			
preserved ~ stimlen + pos	2648.158	0.02308	2640.007	0.408	0.1564	NA	0.052451	NA	NA	NA	NA	NA
					0.0766400							

Model	AIC Delta	AIC	C _w	NagR ²	Intercept	log_stimlen	log_pos	log_freq	I(pos^2)	log_freq	I(pos^2)	log_freq	I(pos^2)	len:I(pos^2)
preserved ~ stimlen * pos	2649.50	2906.00	0.03	0.08	553.95	7418	NA	NA	-	NA	NA	NA	NA	0.0144513
					0.1210960			0.0706179						
preserved ~ stimlen + I(pos^2) + pos	2649.50	1930.37	0.22	0.08	303.74	576	NA	NA	0.1275059	NA	-	NA	NA	NA
					0.0721515						0.0089411			
preserved ~ stimlen	2650.13	5260.13	0.02	0.06	206.13	389	NA	NA	NA	NA	NA	NA	NA	NA
					0.0515498									
preserved ~ 1	2650.82	10100.22	0.00	0.00	0.00	0.00	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen * (I(pos^2) + pos)	2650.12	2805.02	0.04	0.06	149.53	560	NA	NA	-	NA	NA	-	NA	0.0337332e-05
					0.1669018			0.0474492			0.0223361			
preserved ~ pos	2651.33	7504.18	0.00	0.03	0.07	382.32	NA	NA	0.0300026	NA	NA	NA	NA	NA
preserved ~ I(pos^2) + pos	2651.13	3754.10	0.04	0.06	0.47	238.81	NA	NA	0.1393004	NA	-	NA	NA	NA
											0.0128241			

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ pos + log_freq"
```

```
print(BestFLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq
##      2.16048      0.03986      0.10731
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4326 Residual
## Null Deviance:      2649
## Residual Deviance: 2633  AIC: 2639
```

```
# do a median split on frequency to plot hf/lf effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq] <- "hf"
PosDat$freq_bin[PosDat$log_freq < median_freq] <- "lf"
```

```
PosDat$FLPFitted<-fitted(BestFLPModel)
```

```
HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]
```

```
HF_Plot <- plot_len_pos_obs_predicted(HFDat,paste0(CurPat," - ",RunLabel," - High frequency"),"FLPFitted")
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```

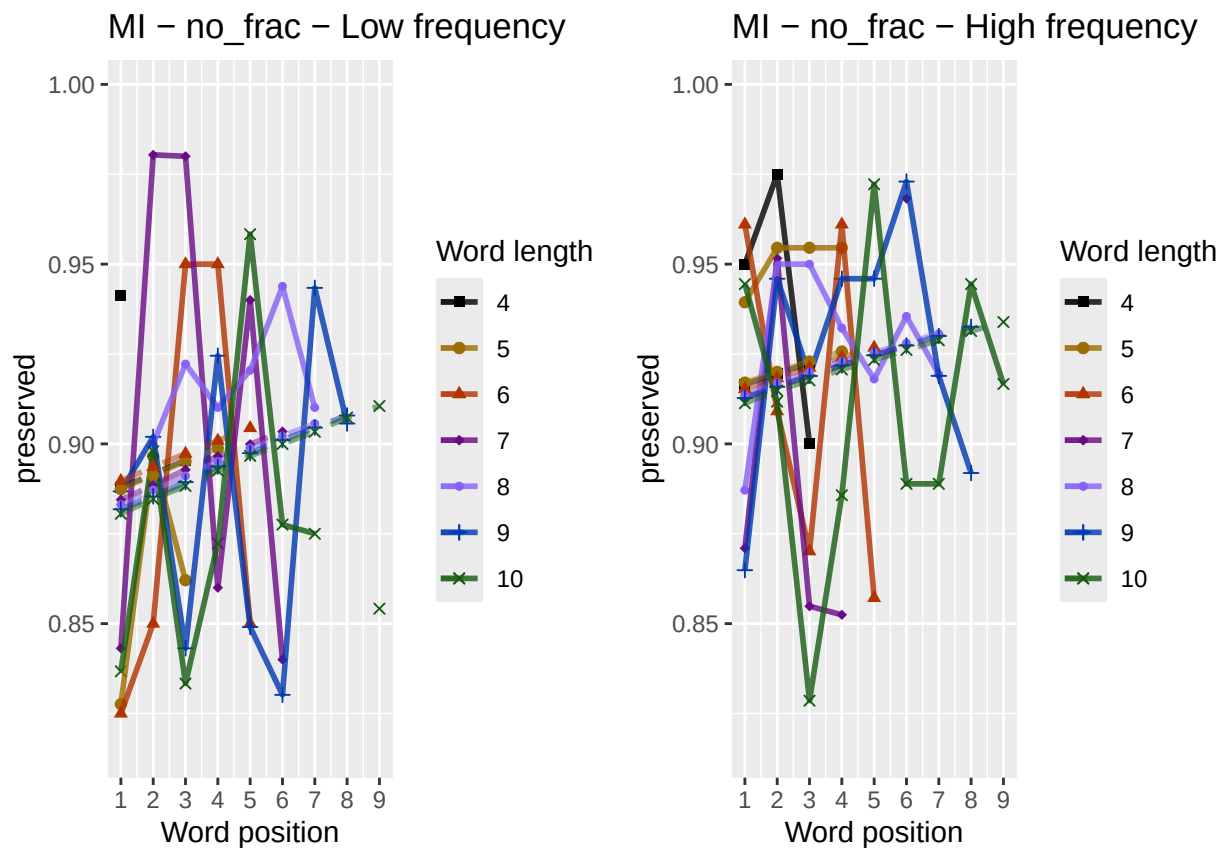
LF_Plot <- plot_len_pos_obs_predicted(LFdat,paste0(CurPat," - ",RunLabel," - Low frequency"),"FLPFitted

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.

library(ggpubr)
Both_Plots <- ggarrange(LF_Plot,HF_Plot) # labels=c("LF","HF",ncol=2)

## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_line()`).
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",
              CurTask,"_frequency_effect_length_pos_wfit.png"),
       device="png",unit="cm",width=30,height=11)
print(Both_Plots)

```



```

# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)

```

```

)
MERes<-TestModels(MEModelEquations,PosDat)

## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.4173      -0.3605
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4327 Residual
## Null Deviance:      2649
## Residual Deviance: 2627  AIC: 2631
## log likelihood:  -1313.373
## Nagelkerke R2:   0.01111588
## % pres/err predicted correctly:  -714.7424
## % of predictable range [ (model-null)/(1-null) ]:  0.005292996
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.08589      0.08778
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4327 Residual
## Null Deviance:      2649
## Residual Deviance: 2638  AIC: 2642
## log likelihood:  -1319.162
## Nagelkerke R2:   0.005294357
## % pres/err predicted correctly:  -716.7749
## % of predictable range [ (model-null)/(1-null) ]:  0.002468297
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      2.69399      -0.05155
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4327 Residual
## Null Deviance:      2649
## Residual Deviance: 2646  AIC: 2650
## log likelihood:  -1323.067
## Nagelkerke R2:   0.00135807
## % pres/err predicted correctly:  -718.1114
## % of predictable range [ (model-null)/(1-null) ]:  0.0006108147

```

```

## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.296
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4328 Residual
## Null Deviance: 2649
## Residual Deviance: 2649 AIC: 2651
## log likelihood: -1324.413
## Nagelkerke R2: 0
## % pres/err predicted correctly: -718.5509
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 2.18232 0.03003
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4327 Residual
## Null Deviance: 2649
## Residual Deviance: 2647 AIC: 2651
## log likelihood: -1323.681
## Nagelkerke R2: 0.000738674
## % pres/err predicted correctly: -718.3029
## % of predictable range [ (model-null)/(1-null) ]: 0.0003446514
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 2.01182 -0.01282 0.13940
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4326 Residual
## Null Deviance: 2649
## Residual Deviance: 2646 AIC: 2652
## log likelihood: -1322.992
## Nagelkerke R2: 0.001433848
## % pres/err predicted correctly: -718.0566
## % of predictable range [ (model-null)/(1-null) ]: 0.0006869882
## *****

```



```

BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                             CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	2630.747	0.00000	1.000000	0.996785	3.011115	2.417337	NA	-	NA	NA	NA
								0.3605046			
preserved ~ CumPres	2642.324	11.57730	0.003062	0.003052	0.005294	1.085892	0.08778	NA	NA	NA	NA
preserved ~ stimlen	2650.135	19.38805	0.000061	0.000061	0.001358	2.693990	NA	NA	NA	NA	-
											0.0515498
preserved ~ 1	2650.822	20.07956	0.000043	0.000043	0.000000	2.295744	NA	NA	NA	NA	NA
preserved ~ pos	2651.362	20.61582	0.000033	0.000033	0.000738	2.182324	NA	NA	NA	0.0300267	NA
preserved ~ (I(pos^2) + pos)	2651.984	21.23782	0.000024	0.000024	0.001433	2.011818	NA	NA	-	0.1393994	NA
									0.0128241		

```

if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                      family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                rep(BestMEModelFormulaRnd,RandomSamples))
  AICValues <- c(BestMEModel$aic,RndModelAIC)
  BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
}

```

```

BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
BestMEModelRndDF <- rbind(BestMEModelRndDF,
                           data.frame(Name=c("Random average"),
                                       AIC=c(mean(RndModelAIC))))
BestMEModelRndDF <- rbind(BestMEModelRndDF,
                           data.frame(Name=c("Random SD"),
                                       AIC=c(sd(RndModelAIC))))

write.csv(BestMEModelRndDF,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_best_main_effects_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary, paste0(TablesDir, CurPat, "_",
                                         RunLabel, "_", CurTask,
                                         "_syllable_component_summary.csv"), row.names = FALSE)
kable(syll_component_summary)

```

syll_component	MeanPres	N
1	0.9097345	565
O	0.8761239	2002
P	0.7647059	34
S	0.7888446	251
V	0.9756263	1477

```

# main effects models for data without satellite positions

keep_components = c("0", "V", "1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen, stim, pos,
                             preserved, syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data, MEModelEquations)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.5263      -0.4151
##
## Degrees of Freedom: 4043 Total (i.e. Null); 4042 Residual
## Null Deviance:      2310

```

```

## Residual Deviance: 2289 AIC: 2293
## log likelihood: -1144.511
## Nagelkerke R2: 0.01207198
## % pres/err predicted correctly: -610.0022
## % of predictable range [ (model-null)/(1-null) ]: 0.005688921
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 2.24309 0.07068
##
## Degrees of Freedom: 4043 Total (i.e. Null); 4042 Residual
## Null Deviance: 2310
## Residual Deviance: 2305 AIC: 2309
## log likelihood: -1152.466
## Nagelkerke R2: 0.003061851
## % pres/err predicted correctly: -612.6585
## % of predictable range [ (model-null)/(1-null) ]: 0.001366257
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 2.78368 -0.04908
##
## Degrees of Freedom: 4043 Total (i.e. Null); 4042 Residual
## Null Deviance: 2310
## Residual Deviance: 2308 AIC: 2312
## log likelihood: -1154.116
## Nagelkerke R2: 0.001188786
## % pres/err predicted correctly: -613.1827
## % of predictable range [ (model-null)/(1-null) ]: 0.0005130621
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.404
##
## Degrees of Freedom: 4043 Total (i.e. Null); 4043 Residual
## Null Deviance: 2310
## Residual Deviance: 2310 AIC: 2312
## log likelihood: -1155.162
## Nagelkerke R2: -5.102039e-16

```

```
## % pres/err predicted correctly: -613.498
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 2.09283 -0.01798 0.17257
##
## Degrees of Freedom: 4043 Total (i.e. Null); 4041 Residual
## Null Deviance: 2310
## Residual Deviance: 2307 AIC: 2313
## log likelihood: -1153.728
## Nagelkerke R2: 0.00162936
## % pres/err predicted correctly: -613.0305
## % of predictable range [ (model-null)/(1-null) ]: 0.0007607493
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 2.33513 0.01811
##
## Degrees of Freedom: 4043 Total (i.e. Null); 4042 Residual
## Null Deviance: 2310
## Residual Deviance: 2310 AIC: 2314
## log likelihood: -1154.932
## Nagelkerke R2: 0.000261287
## % pres/err predicted correctly: -613.4257
## % of predictable range [ (model-null)/(1-null) ]: 0.0001176746
## *****
```

```
write.csv(SimpSyllMEAICSummary,
  paste0(TablesDir, CurPat, "_", RunLabel, "_",
    CurTask,
    "_OV1_main_effects_model_summary.csv"),
  row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	2293.022	0.00000	1.0000000	0.9994513	0.0120722	2.526299	NA	-	NA	NA	NA
preserved ~ CumPres	2308.932	15.91006	0.0003509	0.0003507	0.0030612	2.243091	0.0706773	NA	NA	NA	NA
preserved ~ stimlen	2312.232	19.20967	0.0000674	0.0000674	0.0011888	2.783683	NA	NA	NA	NA	-
preserved ~ 1	2312.325	19.30245	0.0000643	0.0000643	0.0000000	2.404387	NA	NA	NA	NA	0.0490828

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErrI(pos^2)	pos	stimlen
preserved ~ (I(pos^2) + pos)	2313.4520	43379	0.0000365	0.0000365	0.0016292	1.092830	NA	NA	- 0.1725716	NA
									0.0179798	
preserved ~ pos	2313.8620	84256	0.0000298	0.0000298	0.0002612	3.335133	NA	NA	NA	0.0181131
										NA

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0","V")
OVDData <- PosDat[PosDat$syll_component %in% keep_components,]
OVDData <- OVDData %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OVDData$CumPres <- CalcCumPres(OVDData)
OVDData$CumErr <- CalcCumErrFromPreserved(OVDData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVDData,MEModelEquations)
```

```
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.4786      -0.2704
##
## Degrees of Freedom: 3478 Total (i.e. Null); 3477 Residual
## Null Deviance:      1967
## Residual Deviance: 1962 AIC: 1966
## log likelihood: -981.2422
## Nagelkerke R2: 0.003200673
## % pres/err predicted correctly: -519.8329
## % of predictable range [ (model-null)/(1-null) ]: 0.001533209
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.29196      0.06634
##
## Degrees of Freedom: 3478 Total (i.e. Null); 3477 Residual
## Null Deviance:      1967
## Residual Deviance: 1964 AIC: 1968
## log likelihood: -982.0795
## Nagelkerke R2: 0.002087529
## % pres/err predicted correctly: -520.1432
```

```

## % of predictable range [ (model-null)/(1-null) ]: 0.0009383712
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.42
##
## Degrees of Freedom: 3478 Total (i.e. Null); 3478 Residual
## Null Deviance: 1967
## Residual Deviance: 1967 AIC: 1969
## log likelihood: -983.6486
## Nagelkerke R2: 0
## % pres/err predicted correctly: -520.6327
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 2.79176 -0.04817
##
## Degrees of Freedom: 3478 Total (i.e. Null); 3477 Residual
## Null Deviance: 1967
## Residual Deviance: 1966 AIC: 1970
## log likelihood: -982.7829
## Nagelkerke R2: 0.00115187
## % pres/err predicted correctly: -520.3799
## % of predictable range [ (model-null)/(1-null) ]: 0.000484498
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 2.12317 -0.02039 0.18291
##
## Degrees of Freedom: 3478 Total (i.e. Null); 3476 Residual
## Null Deviance: 1967
## Residual Deviance: 1964 AIC: 1970
## log likelihood: -982.1853
## Nagelkerke R2: 0.001946821
## % pres/err predicted correctly: -520.1562
## % of predictable range [ (model-null)/(1-null) ]: 0.0009134205
## *****
## model index: 4

```

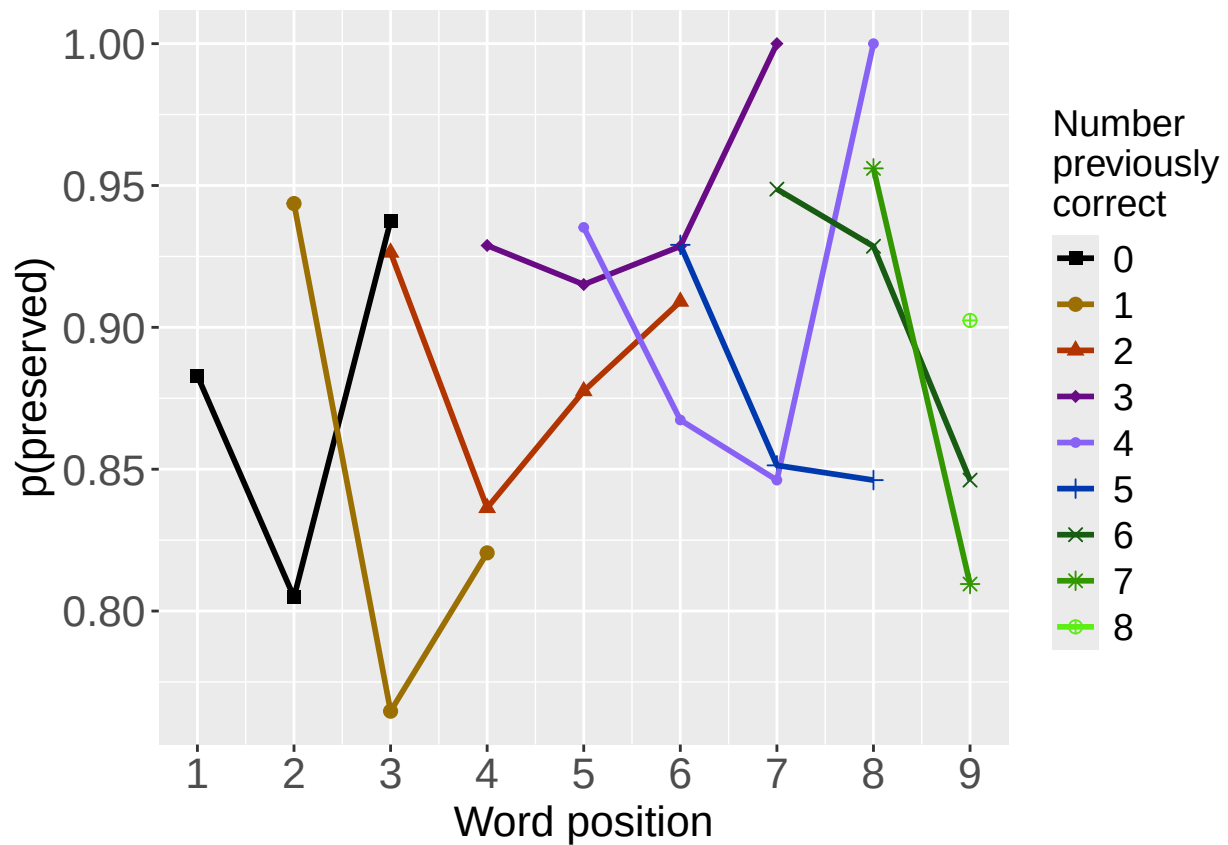
```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
## 2.391201      0.007632
##
## Degrees of Freedom: 3478 Total (i.e. Null); 3477 Residual
## Null Deviance: 1967
## Residual Deviance: 1967 AIC: 1971
## log likelihood: -983.6115
## Nagelkerke R2: 4.929421e-05
## % pres/err predicted correctly: -520.6211
## % of predictable range [ (model-null)/(1-null) ]: 2.221524e-05
## *****
```

```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1966.484	0.000000	0.000000	0.469714	0.003200	2.478625	NA	-	NA	NA	NA
preserved ~ CumPres	1968.159	1.674544	0.432889	0.203334	0.002087	2.291963	0.0663387	NA	NA	NA	NA
preserved ~ 1	1969.292	2.812720	0.245033	0.115095	0.000000	0.420368	NA	NA	NA	NA	NA
preserved ~ stimlen	1969.566	3.081460	0.214223	0.100624	0.001151	2.791758	NA	NA	NA	NA	-
preserved ~ (I(pos^2) + pos)	1970.373	3.886160	0.143262	0.067292	0.001946	2.123170	NA	NA	-	0.1829052	NA
preserved ~ pos	1971.224	4.738650	0.093543	0.043938	0.000049	2.391202	NA	NA	NA	0.0076324	NA

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

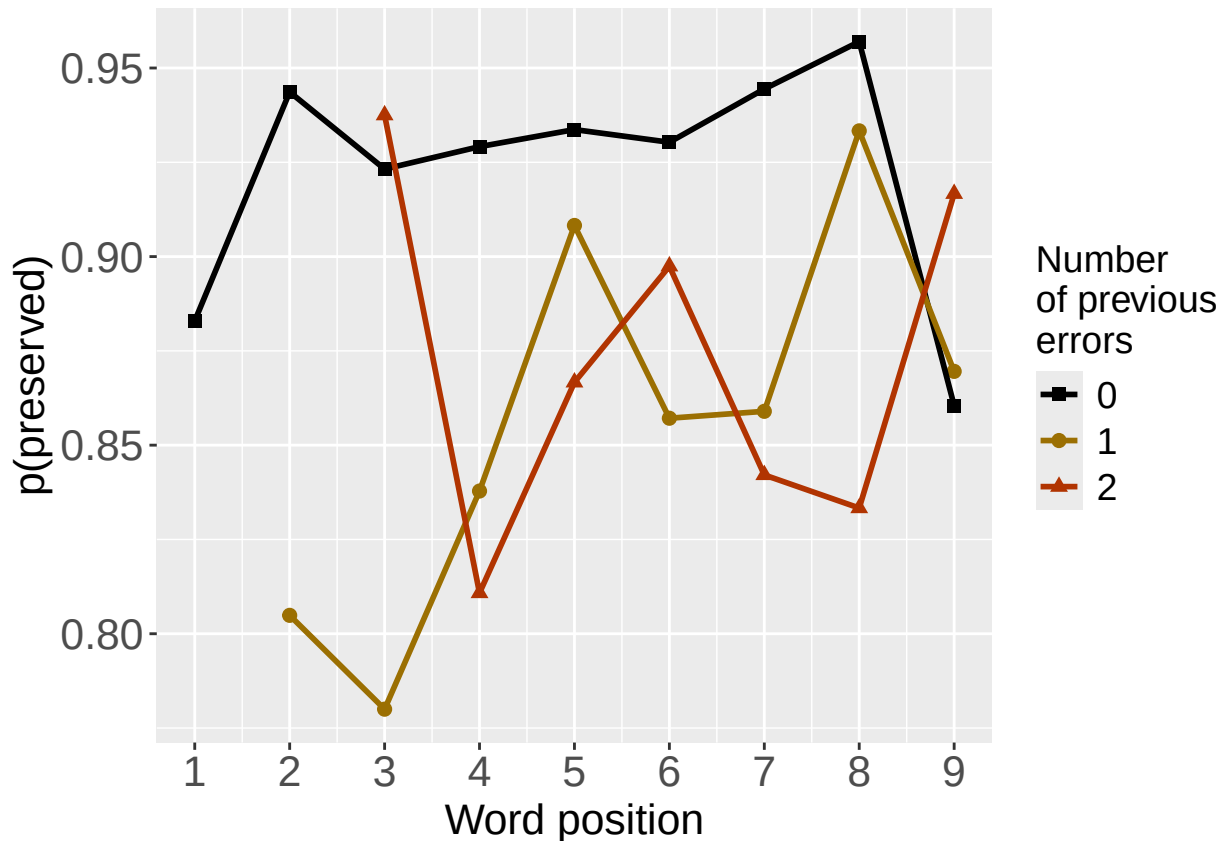
```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tiff"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

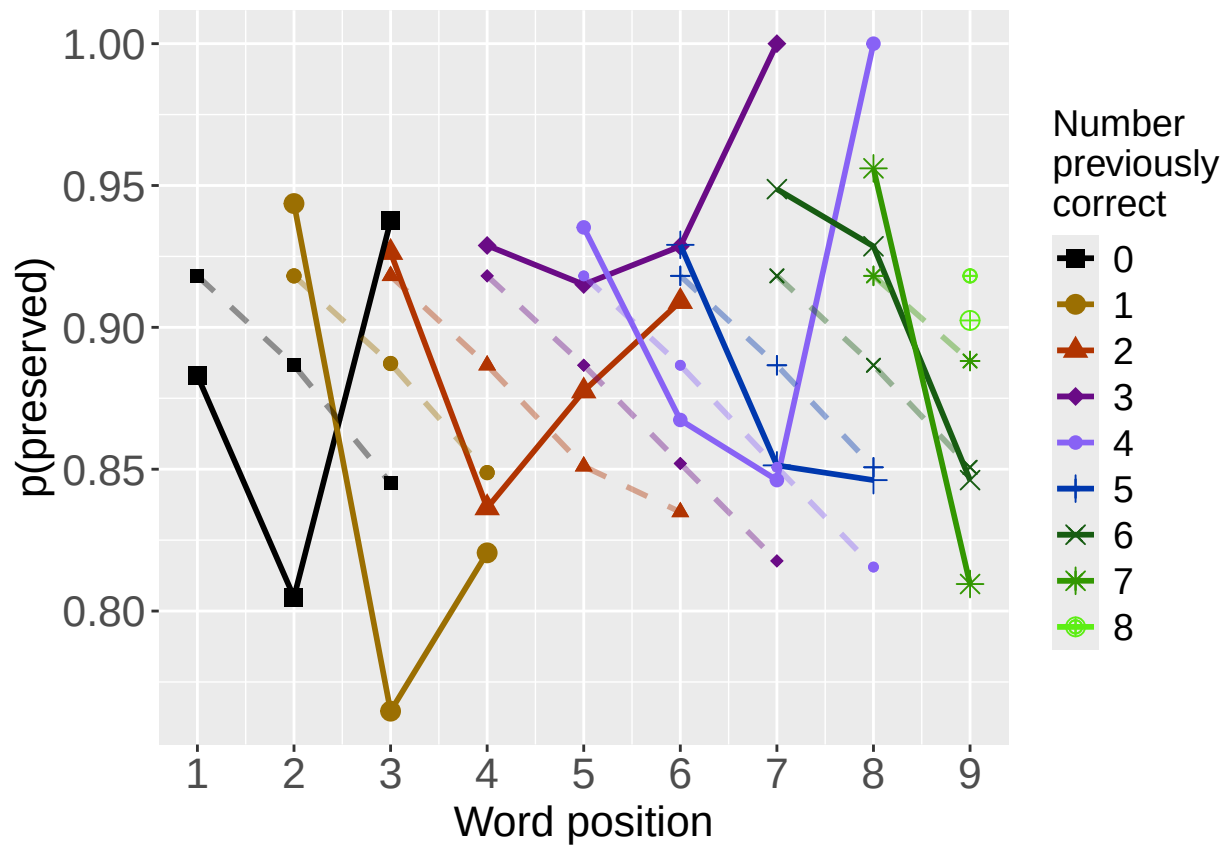
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tiff"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

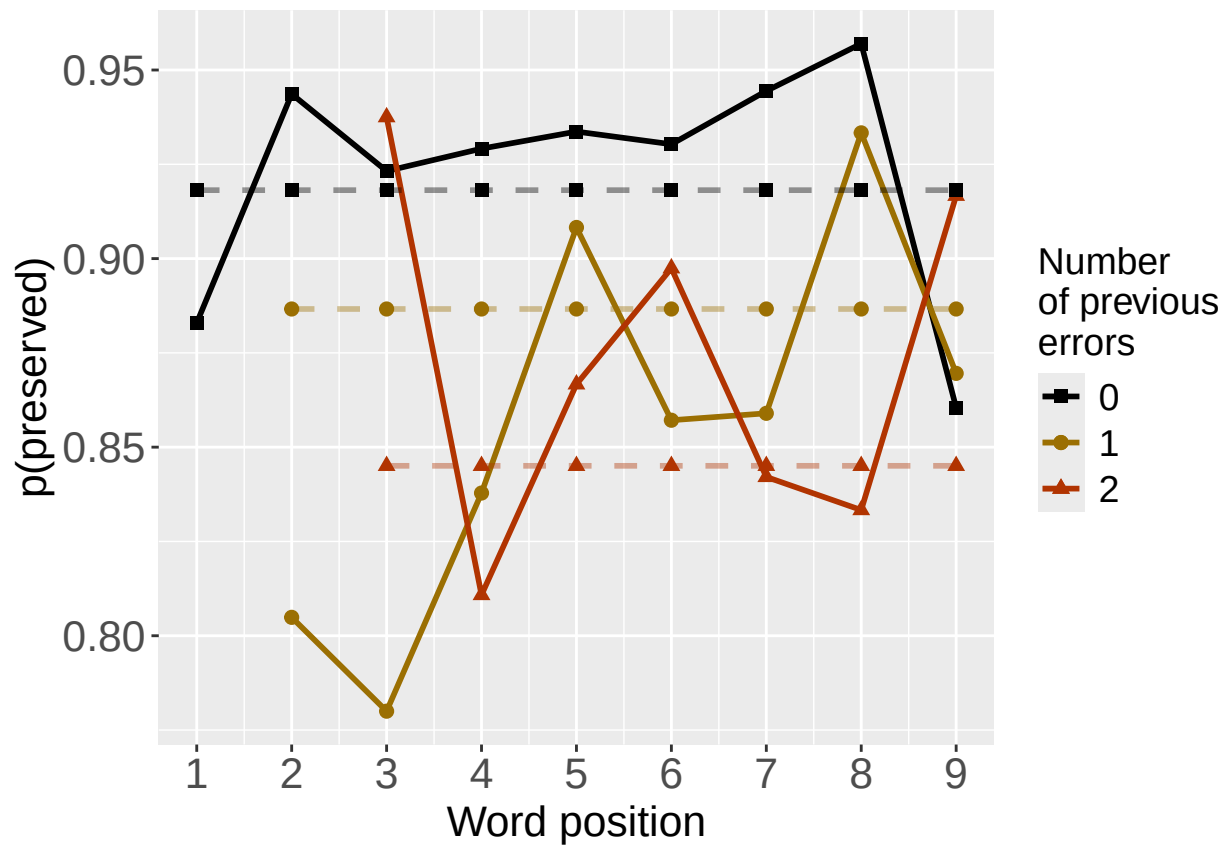
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)         pos
##    1.95972    -0.46651    -0.01247     0.19335
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4325 Residual
## Null Deviance:      2649
## Residual Deviance: 2615  AIC: 2623
## log likelihood:  -1307.636
## Nagelkerke R2:  0.0168703
## % pres/err predicted correctly:  -712.8534
## % of predictable range [ (model-null)/(1-null) ]:  0.007918174
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.4173      -0.3605
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4327 Residual
## Null Deviance:      2649
## Residual Deviance: 2627  AIC: 2631
## log likelihood:  -1313.373
## Nagelkerke R2:  0.01111588
## % pres/err predicted correctly:  -714.7424
## % of predictable range [ (model-null)/(1-null) ]:  0.005292996
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.01182      -0.01282      0.13940
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4326 Residual
## Null Deviance:      2649
## Residual Deviance: 2646  AIC: 2652
## log likelihood:  -1322.992
## Nagelkerke R2:  0.001433848
## % pres/err predicted correctly:  -718.0566
## % of predictable range [ (model-null)/(1-null) ]:  0.0006869882
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	2623.272	0.00000	1.0000000	0.9767333	0.0168703	1.959720	-0.4665140	-0.0124742	0.1933503
preserved ~ CumErr	2630.747	7.47443	0.0238204	0.0232661	0.0111159	2.417337	-0.3605046	NA	NA
preserved ~ I(pos^2) + pos	2651.984	28.71225	0.0000006	0.0000006	0.0014338	2.011818	NA	-0.0128241	0.1393994

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 2.4173 -0.3605
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4327 Residual
## Null Deviance: 2649
## Residual Deviance: 2627 AIC: 2631
## log likelihood: -1313.373
## Nagelkerke R2: 0.01111588
## % pres/err predicted correctly: -714.7424
## % of predictable range [ (model-null)/(1-null) ]: 0.005292996
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr stimlen
## 2.61056 -0.34846 -0.02555
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4326 Residual
## Null Deviance: 2649
## Residual Deviance: 2626 AIC: 2632
## log likelihood: -1313.059
## Nagelkerke R2: 0.01143123
## % pres/err predicted correctly: -714.6833
## % of predictable range [ (model-null)/(1-null) ]: 0.005375072
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      2.69399      -0.05155
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4327 Residual
## Null Deviance:      2649
## Residual Deviance: 2646  AIC: 2650
## log likelihood:  -1323.067
## Nagelkerke R2:   0.00135807
## % pres/err predicted correctly:  -718.1114
## % of predictable range [ (model-null)/(1-null) ]:  0.0006108147
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr	2630.747	0.000000	1.0000000	0.665048	0.0111159	2.417337	- 0.3605046	NA
preserved ~ CumErr + stimlen	2632.119	1.371989	0.5035892	0.334911	0.0114312	2.610563	- 0.3484632	- 0.0255454
preserved ~ stimlen	2650.135	19.388051	0.0000617	0.000041	0.0013581	2.693990	NA	- 0.0515498

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.18797      -0.38108      0.09896
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4326 Residual
## Null Deviance:      2649
## Residual Deviance: 2614  AIC: 2620
## log likelihood:  -1306.946
## Nagelkerke R2:   0.01756147
## % pres/err predicted correctly:  -712.4026
## % of predictable range [ (model-null)/(1-null) ]:  0.008544734
## *****
## model index:  1
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.4173      -0.3605
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4327 Residual
## Null Deviance:      2649
## Residual Deviance: 2627  AIC: 2631
## log likelihood:  -1313.373
## Nagelkerke R2:  0.01111588
## % pres/err predicted correctly:  -714.7424
## % of predictable range [ (model-null)/(1-null) ]:  0.005292996
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.08589      0.08778
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4327 Residual
## Null Deviance:      2649
## Residual Deviance: 2638  AIC: 2642
## log likelihood:  -1319.162
## Nagelkerke R2:  0.005294357
## % pres/err predicted correctly:  -716.7749
## % of predictable range [ (model-null)/(1-null) ]:  0.002468297
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	2619.892	0.00000	1.0000000	0.9956111	0.0175615	2.187968	- 0.3810838	0.0989637
preserved ~ CumErr	2630.747	10.85469	0.0043947	0.0043755	0.0111159	2.417337	- 0.3605046	NA
preserved ~ CumPres	2642.324	22.43208	0.0000135	0.0000134	0.0052944	2.085892	NA	0.0877800

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      2.12481      -0.46722      0.08723
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4326 Residual
## Null Deviance:      2649
## Residual Deviance: 2617 AIC: 2623
## log likelihood: -1308.279
## Nagelkerke R2: 0.01622588
## % pres/err predicted correctly: -712.9313
## % of predictable range [ (model-null)/(1-null) ]: 0.007809888
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.4173      -0.3605
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4327 Residual
## Null Deviance:      2649
## Residual Deviance: 2627 AIC: 2631
## log likelihood: -1313.373
## Nagelkerke R2: 0.01111588
## % pres/err predicted correctly: -714.7424
## % of predictable range [ (model-null)/(1-null) ]: 0.005292996
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      2.18232      0.03003
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4327 Residual
## Null Deviance:      2649
## Residual Deviance: 2647 AIC: 2651
## log likelihood: -1323.681
## Nagelkerke R2: 0.000738674
## % pres/err predicted correctly: -718.3029
## % of predictable range [ (model-null)/(1-null) ]: 0.0003446514
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr	2622.559	0.000000	1.000000	0.9835999	0.0162259	2.124808	-	0.0872273
+ pos							0.4672235	
preserved ~ CumErr	2630.747	8.187925	0.0166730	0.0163996	0.0111159	2.417337	-	NA
							0.3605046	
preserved ~ pos	2651.362	28.803744	0.0000006	0.0000005	0.0007387	2.182324	NA	0.0300267

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_main_effects_plus_one_model_summary.csv"), row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + CumPres	2619.890	0.000000	1.000000	0.995611	0.017561	2.187968	-	NA	NA	NA	0.0989637
							0.3810838				
preserved ~ CumErr + pos	2622.559	0.000000	1.000000	0.9835999	0.0162259	2.124808	-	NA	0.0872273	NA	NA
							0.4672235				
preserved ~ CumErr + I(pos^2) + pos	2623.270	0.000000	1.000000	0.9767338	0.0168703	2.3959720	-	-	0.1933503	NA	NA
							0.4665140	0.0124742			
preserved ~ CumErr	2630.747	4.744300	0.0238201	0.0232660	0.0111159	2.417337	-	NA	NA	NA	NA
							0.3605046				
preserved ~ CumErr	2630.747	0.000000	1.000000	0.6650480	0.0111159	2.417337	-	NA	NA	NA	NA
							0.3605046				
preserved ~ CumErr	2630.747	0.854691	0.0043947	0.0043755	0.0111159	2.417337	-	NA	NA	NA	NA
							0.3605046				
preserved ~ CumErr	2630.748	8.187925	0.0166730	0.0163996	0.0111159	2.417337	-	NA	NA	NA	NA
							0.3605046				
preserved ~ CumErr + stimlen	2632.119	3.719890	0.5035892	0.2334910	0.0114312	2.610563	-	NA	NA	-	NA
							0.3484632			0.0255454	
preserved ~ CumPres	2642.322	22.432078	0.0000013	0.0000013	0.0052941	2.085892	NA	NA	NA	NA	0.0877800
preserved ~ stimlen	2650.135	9.388051	0.0000617	0.0000410	0.0013581	2.693990	NA	NA	NA	-	NA
										0.0515498	
preserved ~ pos	2651.362	8.803744	0.0000006	0.0000005	0.0007387	2.182324	NA	NA	0.0300267	NA	NA
preserved ~ I(pos^2) + pos	2651.982	8.712249	0.0000006	0.0000006	0.0014338	2.011818	NA	-	0.1393994	NA	NA
								0.0128241			

```
# explore influence of frequency and length

if(grepl("stimlen", BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq")
  )
}else if(grepl("log_freq", BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
```

```

    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq"),
    paste0(BestModelFormulaL2, " + stimlen"),
    paste0(BestModelFormulaL2, " + stimlen + log_freq")
  )
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      log_freq
##      2.17707      -0.34772      0.10435      0.09491
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4325 Residual
## Null Deviance:      2649
## Residual Deviance: 2603 AIC: 2611
## log likelihood: -1301.292
## Nagelkerke R2: 0.02321525
## % pres/err predicted correctly: -710.4615
## % of predictable range [ (model-null)/(1-null) ]: 0.01124239
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen      log_freq
##      2.50173      -0.33310      0.11621      -0.04654      0.08505
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4324 Residual
## Null Deviance:      2649
## Residual Deviance: 2601 AIC: 2611
## log likelihood: -1300.413
## Nagelkerke R2: 0.02409355
## % pres/err predicted correctly: -710.1819
## % of predictable range [ (model-null)/(1-null) ]: 0.01163098
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen
##      2.68897      -0.35235      0.11862      -0.07224
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4325 Residual
## Null Deviance:      2649
## Residual Deviance: 2609 AIC: 2617
## log likelihood:  -1304.659
## Nagelkerke R2:  0.01985041
## % pres/err predicted correctly:  -711.6436
## % of predictable range [ (model-null)/(1-null) ]:  0.009599522
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.18797      -0.38108      0.09896
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4326 Residual
## Null Deviance:      2649
## Residual Deviance: 2614 AIC: 2620
## log likelihood:  -1306.946
## Nagelkerke R2:  0.01756147
## % pres/err predicted correctly:  -712.4026
## % of predictable range [ (model-null)/(1-null) ]:  0.008544734
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.296
##
## Degrees of Freedom: 4328 Total (i.e. Null);  4328 Residual
## Null Deviance:      2649
## Residual Deviance: 2649 AIC: 2651
## log likelihood:  -1324.413
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -718.5509
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,

```

```

      row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres	log_freq	stimlen
preserved ~ CumErr + CumPres + log_freq	2610.586	0.000000	0.000000	0.517974	0.602321	52177067	-	0.104345	0.094912	NA
							0.3477168			
preserved ~ CumErr + CumPres + stimlen + log_freq	2610.826	0.240818	0.886557	0.445921	0.202409	35501726	-	0.116207	0.808505	-
							0.3331006			0.0465411
preserved ~ CumErr + CumPres + stimlen	2617.318	6.732931	0.034510	0.017876	0.001985	0.21688969	-	0.118615	NA	-
							0.3523546			0.0722377
preserved ~ CumErr + CumPres	2619.892	9.307060	0.009527	0.000493	0.001756	15187968	-	0.098963	NA	NA
							0.3810838			
preserved ~ 1	2650.824	40.241316	0.000000	0.000000	0.000000	0.0295744	NA	NA	NA	NA

```

# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

```

```

## Single term deletions
##
## Model:
## preserved ~ CumErr + CumPres + log_freq
##      Df Deviance   AIC
## CumErr    1   2622.8 2628.8
## CumPres    1   2616.9 2622.9
## log_freq    1   2613.9 2619.9
## <none>      2602.6 2610.6

```

```

#####
# Single deletions from best model
#####

```

```

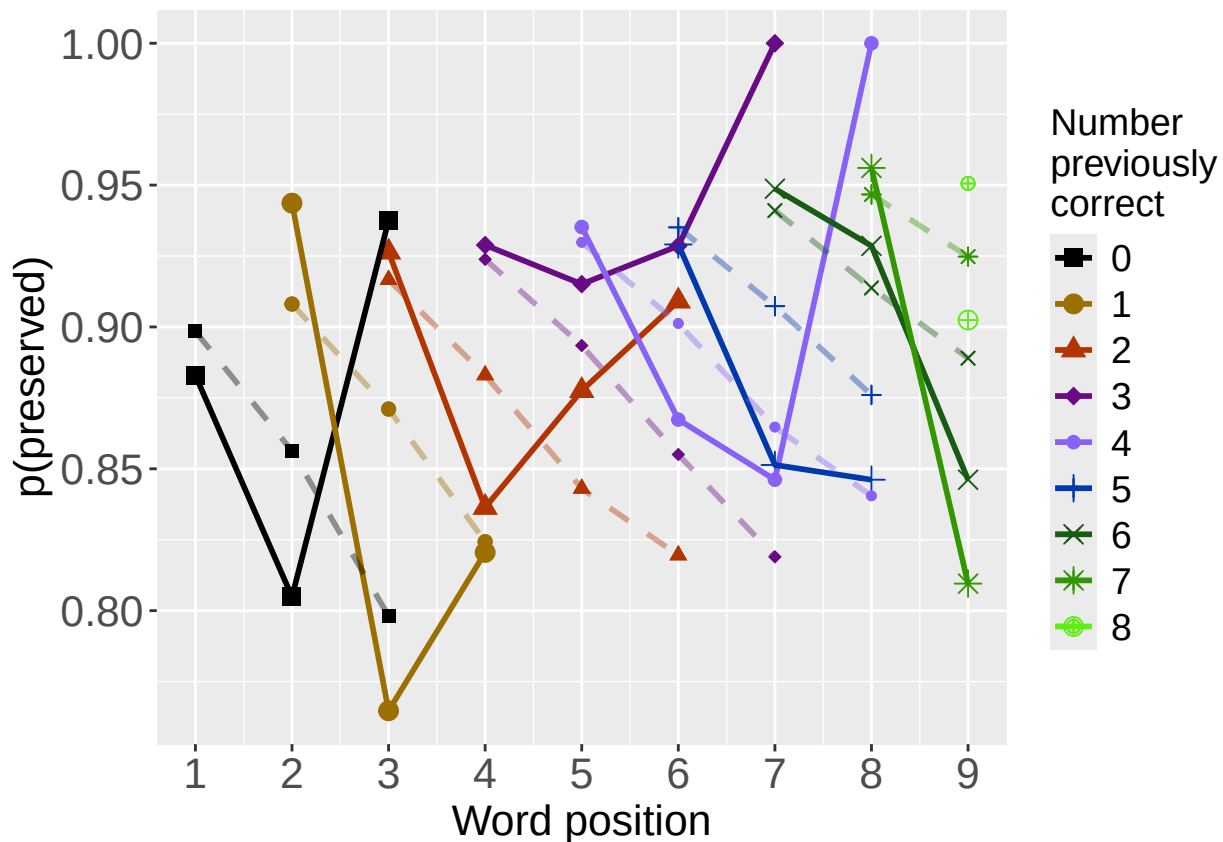
write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                     RunLabel,"_",CurTask,
                                     "_best_model_single_term_deletions.csv"),
          row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```

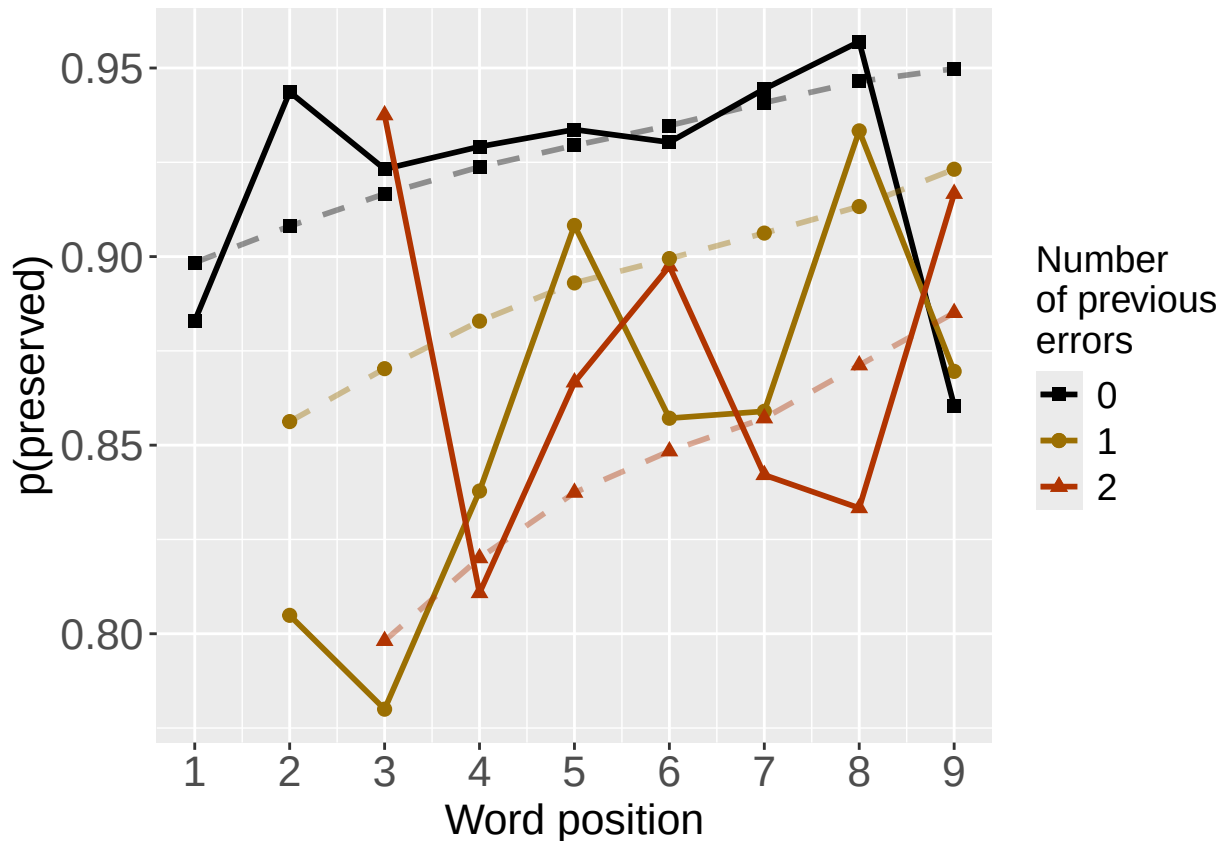


```

PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)

```



```

PlotName<-paste0(FigDir,CurPat,"_",
                  RunLabel,"_",CurTask,
                  "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.4173      -0.3605
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4327 Residual
## Null Deviance:      2649
## Residual Deviance: 2627 AIC: 2631
## log likelihood: -1313.373
## Nagelkerke R2: 0.01111588
## % pres/err predicted correctly: -714.7424
## % of predictable range [ (model-null)/(1-null) ]: 0.005292996

```



```

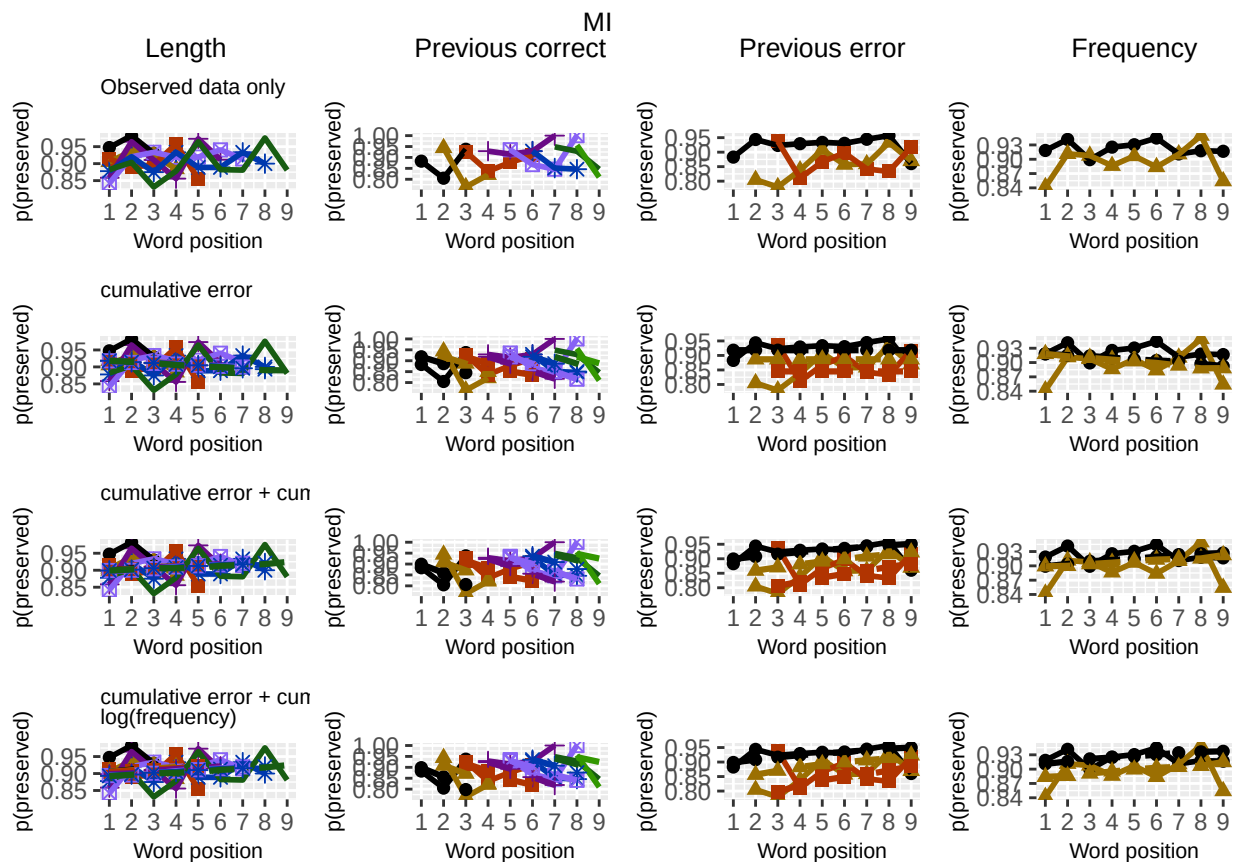
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.18797      -0.38108      0.09896
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4326 Residual
## Null Deviance:      2649
## Residual Deviance: 2614 AIC: 2620
## log likelihood: -1306.946
## Nagelkerke R2: 0.01756147
## % pres/err predicted correctly: -712.4026
## % of predictable range [ (model-null)/(1-null) ]: 0.008544734
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      log_freq
##      2.17707      -0.34772      0.10435      0.09491
##
## Degrees of Freedom: 4328 Total (i.e. Null); 4325 Residual
## Null Deviance:      2649
## Residual Deviance: 2603 AIC: 2611
## log likelihood: -1301.292
## Nagelkerke R2: 0.02321525
## % pres/err predicted correctly: -710.4615
## % of predictable range [ (model-null)/(1-null) ]: 0.01124239
## *****
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

```

```

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot

```



```

DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                     CurTask,"_dominance_analysis_table.csv"),
          row.names = TRUE)
kable(DAContributionAverage)

```

	CumErr	CumPres	log_freq
McFadden	0.0080076	0.0047137	0.0047360
SquaredCorrelation	0.0048738	0.0028679	0.0028832
Nagelkerke	0.0048738	0.0028679	0.0028832
Estrella	0.0049163	0.0028947	0.0029073

	deviance	deviance_explained
CumErr + CumPres + log_freq	2602.585	46.24132
CumErr + CumPres	2613.892	34.93426
CumErr	2626.747	22.07956
null	2648.826	0.00000

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + CumPres + log_freq	1.7457286	100.00000	24.45229
CumErr + CumPres	1.3188580	75.54771	27.79915
CumErr	0.8335604	47.74857	47.74857
null	0.0000000	NA	0.00000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                model deviance deviance_explained percent_explained percent_of_explained_deviance
## CumErr + CumPres + log_freq CumErr + CumPres + log_freq 2602.585          46.24132          1.7457286          100.00000
## CumErr + CumPres              CumErr + CumPres 2613.892          34.93426          1.3188580          75.54771
## CumErr                        CumErr 2626.747          22.07957          0.8335604          47.74857
## null                          null 2648.826          0.00000          0.0000000          NA
##                                increment_in_explained
## CumErr + CumPres + log_freq          24.45229
## CumErr + CumPres                    27.79915
## CumErr                              47.74857
## null                                0.00000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
```

```

NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##           Nagelkerke
## CumErr    0.4587178
## CumPres   0.2699188
## log_freq  0.2713634

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

```

model	p_accounted_for	model_deviance
preserved ~ CumErr	0.3807574	2626.747
preserved ~ CumErr+CumPres	0.4974002	2613.892
preserved ~ CumErr+CumPres+log_freq	0.5081343	2602.585

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_sse_results_table.csv"),row.names = TRUE)
sse_table
```

```
##               model p_accounted_for model_deviance diff_CumErr diff_CumErr+CumPres diff_CumErr+CumPres+log_freq
## 1      preserved ~ CumErr      0.3807574      2626.747      0.0000000      -0.1166428      -0.1273769
## 2      preserved ~ CumErr+CumPres      0.4974002      2613.892      0.1166428      0.0000000      -0.0107341
## 3 preserved ~ CumErr+CumPres+log_freq      0.5081343      2602.585      0.1273769      0.0107341      0.0000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                   CurTask,"_results_report_df.csv"),row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

model	diff_CumErr	diff_CumErr+CumPres	diff_CumErr+CumPres+log_freq
preserved $\sim$ CumErr	0.000000	-0.1166428	-0.1273769
preserved $\sim$ CumErr+CumPres	0.1166428	0.000000	-0.0107341
preserved $\sim$ CumErr+CumPres+log_freq	0.1273769	0.0107341	0.000000